Ecosystem Survival

Checkpoint Document

Computer Science I

Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

Team Members:

Cristian Esteban Castañeda Vargas
Julian Carvajal Garnica
Juan Pablo Angulo Guerrero

Group: 8

Submission: Week 8 - Milestone 1

27 de marzo de 2026

Índice

1. Algorithm Pseudocode

1.1. Greedy Algorithm

The greedy algorithm is used to recommend which species should be attended first in each turn. To achieve this, each species is evaluated based on a risk value that depends on its hunger and population.

```
function GreedyRecommend(species_list):

    best_species <- null
    highest_risk <- -1

    for each species in species_list:

        if species.status != "extinct" and species.status != "safe":

            risk <- species.hunger + (10 - species.population)

            if risk > highest_risk:
                highest_risk <- risk
                best_species <- species

    if best_species == null:
        return null

    return best_species
```

1.2. Backtracking Algorithm

```
function FindMigrationPath(board, row, col, visited, path):

    if board[row][col] == "Z":
        return true

    visited[row][col] <- true

    for each (dr, dc) in 8 directions:

        new_row <- row + dr
```

```
        new_col <- col + dc

        if IsValidCell(board, new_row, new_col, visited):

            append (new_row, new_col) to path

            if FindMigrationPath(board, new_row, new_col, visited, path):
                return true

            remove last element from path

    return false
```

1.3. Validation Function

```
function IsValidCell(board, row, col, visited):

    if row < 0 or row >= 8:
        return false

    if col < 0 or col >= 8:
        return false

    if visited[row][col] == true:
        return false

    if board[row][col] == "X":
        return false

    if board[row][col] is another species:
        return false

    return true
```

2. Data Structure Design

2.1. Linked List

```
struct PathNode {
```

```
    int row;
    int col;
    PathNode* next;
};
```

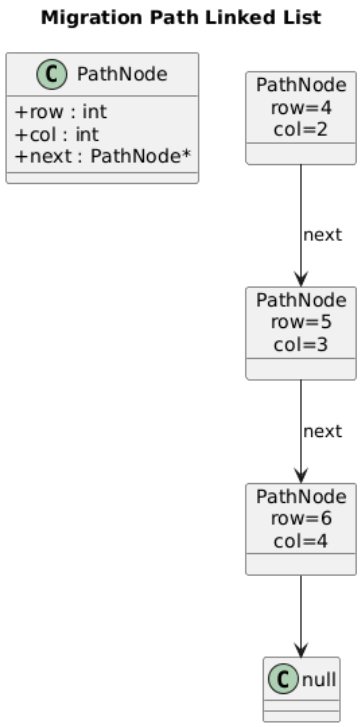

Figura 1: Linked list diagram

2.2. AVL Tree

```
struct AVLNode {
    string symbol;
    string name;
    int population;
    int hunger;
    int height;
    AVLNode* left;
    AVLNode* right;
};
```

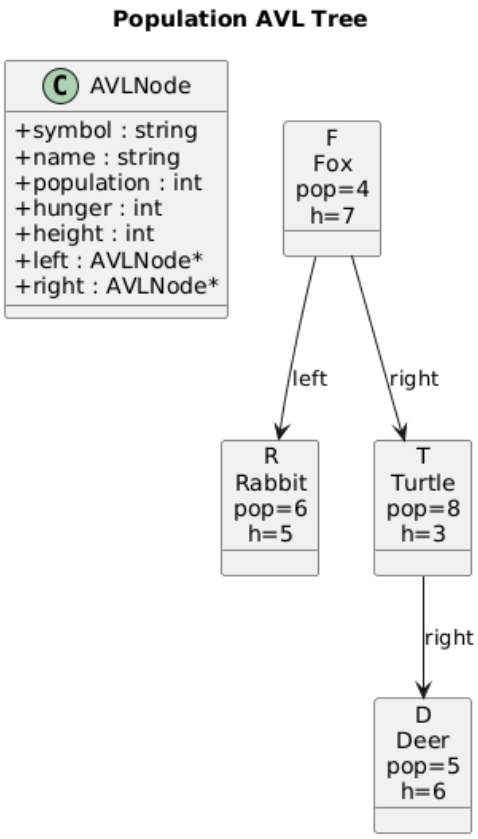

Figura 2: AVL tree diagram

3. JSON File Contract

3.1. Input File

```
{
  "action": "feed",
  "species": "R"
}
```

- `action` defines the type of action (feed or migrate)
- `species` identifies the selected species

3.2. State File

```
{
  "grid_size": 8,
  "turn": 3,
```

```
  "actions_left": 1,
  "board": [...],
  "species": [...],
  "greedy_recommendation": "F",
  "migration_path": [...],
  "game_status": "running"
}
```

- `grid_size` is the size of the grid
- `turn` indicates the current turn
- `actions_left` represents remaining actions
- `board` stores the grid
- `species` contains species data
- `greedy_recommendation` shows the suggested species
- `migration_path` contains the computed path
- `game_status` represents the game state

4. Interface Design (Wireframe)

```
[ R ][   ][ X ][   ]
[   ][ C ][   ][   ]
[   ][   ][ F ][   ]
[ Z ][   ][   ][ D ]
```

5. Work Distribution

Member	Role	Responsibility
Cristian	C++	Data structures implementation
Julian	Algorithms	Greedy and backtracking development
Juan	UI	Pygame interface development